
BPM: Beat-Driven Program Modeling for Automatic Music Mixing

Hongyang Du¹ Ruijie Ye¹ Yuehuan Qiu¹

Abstract

We propose a hybrid learning–signal-processing framework for automatic music transition generation between two songs. Instead of directly synthesizing audio—a task that is computationally demanding and often unstable for neural models—we train a transformer to predict a set of time-varying Digital Signal Processing(DSP) control parameters that define a professional DJ-style transition. Using MERT as a high-level music encoder, we extract frame-level structural embeddings of the ending segment of the outgoing track and the beginning segment of the incoming track. The transformer performs masked-middle prediction to generate transition embeddings, which are decoded into DSP curves including crossfade envelopes, multi-band EQ gains, high-pass/low-pass filter sweeps, and transition duration. A deterministic DSP renderer then applies these parameters to the original audio, ensuring high-fidelity, beat-aligned, and musically coherent transitions while keeping the system interpretable and stable. Our approach leverages the strengths of both deep learning and classic audio engineering, achieving artifact-free transitions without training a full audio generative model. Code, data are available at this https URL <https://github.com/DafeiQiu123/automixer>.

1. Introduction

Music is commonly regarded as a special kind of “language” that conveys emotion and structure. Although recent music-pretrained models such as MERT have demonstrated strong capabilities on classification and transcription tasks (Yizhi et al., 2023), they remain limited for generation tasks such as music fusion and mashup creation. These limitations stem from two issues: (1) the absence of training data for intermediate variations (e.g., transitions, interludes), and (2) the encoder-only design of current music representation models, which prevents direct synthesis of new musical material.

To address these challenges, we adopt a *formal modeling* perspective for music structure and propose a framework that maps continuous audio representations to discrete mixing

instructions. Specifically, we use MERT with beat-position information as the encoder and represent the decoder-side generation space using a discrete sequence of DSP operations. This design allows us to model music transition generation as predicting a structured DSP instruction sequence, rather than synthesizing raw audio. We train and evaluate our framework using the FMA dataset (Defferrard et al., 2017).

2. Related Work

Recent advances in self-supervised learning have led to powerful general-purpose encoders for audio and music. Speech models such as wav2vec 2.0 (Baevski et al., 2020) and HuBERT (Hsu et al., 2021) showed that masked prediction over discrete acoustic units can yield highly transferable representations, and this idea has been extended to music through models like MERT (Yizhi et al., 2023), which is trained at scale on raw music audio using teacher-guided masked prediction to capture both acoustic and musical structure. These foundation models are typically used as frozen feature extractors with task-specific heads for tagging, transcription, or other music information retrieval tasks, and most existing work processes each audio clip independently. Our work leverages MERT as a pretrained music encoder and focuses on how to effectively fuse its high-level representations across multiple music inputs into a unified embedding. This shifts the problem from learning representations from scratch to designing a principled fusion mechanism on top of a strong music foundation model, enabling more flexible and efficient use of MERT for downstream music understanding.

3. Data

All experiments are conducted on a subset of FMA: A DATASET FOR MUSIC ANALYSIS (Defferrard et al., 2017). We do not use any additional proprietary or external data beyond this corpus.

4. Methodology

Our initial plan was to build a dedicated decoder module for music generation from scratch, and to reconstruct the input audio in a fully serialized way. However, our investigation showed that training a high-quality neural decoder from scratch would require far more data and computation than

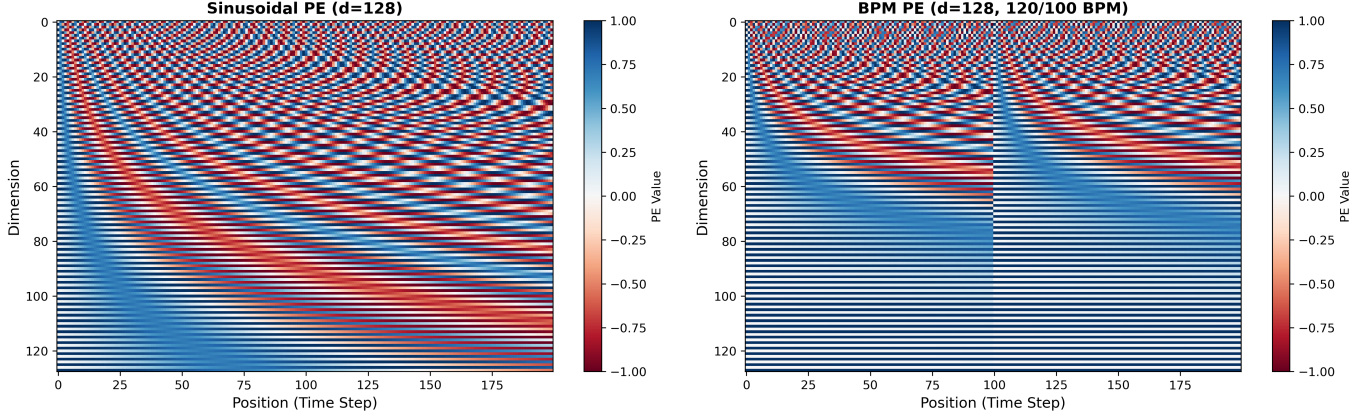


Figure 1. Comparison of positional encoding patterns. Left: the standard sinusoidal positional encoding exhibits smoothly increasing frequencies across dimensions. Right: our DualBPM positional encoding introduces a tempo-dependent frequency shift at the segment boundary (120 BPM $\rightarrow$ 100 BPM), producing distinct structure while preserving continuity in early dimensions.

we currently have access to. Therefore, we turn to a more lightweight DSP-based formal encoding scheme, and reformulate music generation as a conditional modeling problem of predicting a sequence of pure DSP automatic mixing codes.

4.1. DSP Representation and Basic Mathematical Form

DSP can be viewed as an automatic mixing script composed purely of digital signal processing (DSP) primitives (Oppenheim & Schaffer, 2010). Its core idea is to apply a sequence of parameterized DSP operators to the raw audio signal $x[n]$ to obtain the mixed output signal $y[n]$. Formally, a DSP sequence can be written as:

$$\mathcal{D} = \{(t_i, \mathbf{p}_i, o_i)\}_{i=1}^L,$$

where t_i denotes the time at which the i -th operation takes effect (defined either in the beat domain or in the sample domain), o_i is the type of the i -th DSP operator (e.g., gain control, equalization, compression, reverberation), and $\mathbf{p}_i \in \mathbb{R}^K$ is the corresponding parameter vector (such as gain value, center frequency, Q-factor, compression threshold, attack/release time, and reverberation decay time). The overall mixing process can be abstracted as a composition of functions:

$$y[n] = (f_{o_L, \mathbf{p}_L} \circ f_{o_{L-1}, \mathbf{p}_{L-1}} \circ \cdots \circ f_{o_1, \mathbf{p}_1})(x)[n],$$

where $f_{o_i, \mathbf{p}_i}$ denotes the DSP transformation determined by operator type o_i and parameter vector $\mathbf{p}_i$. The learning objective of the model is to predict a suitable DSP sequence $\mathcal{D}$ from the input music representation (e.g., the MERT encoding $\mathbf{h}$), so that a downstream DSP engine can automatically perform mixing and mashup synthesis.

4.2. BPM Definition and Beat Position Encoding

BPM stands for *Beats Per Minute*, and measures the tempo of music. If the time interval between two consecutive beats is T seconds, the corresponding BPM value is defined as:

$$\text{BPM} = \frac{60}{T}.$$

In discrete time, if the sampling rate is f_s and the number of samples between two beats is N_{beat} , then $T = N_{\text{beat}}/f_s$, and thus

$$\text{BPM} = \frac{60f_s}{N_{\text{beat}}}.$$

BPM provides a macro-level measure of musical tempo and naturally defines a beat-based time grid. In this work, we explicitly introduce BPM-based beat position encoding (BPM Position Encoding) on top of the encoder outputs of MERT. Concretely, we map the beat index (bar location, beat number, and intra-beat relative phase) into a positional vector $\mathbf{e}_{\text{BPM}}$, and fuse it with the original temporal positional encoding. This helps the model align the encoded musical structure with rhythmic information.

Since accurate BPM labels are expensive to annotate, we estimate BPM automatically and inject several Transformer layers with BPM Position Encoding. We further conduct ablation studies to evaluate the benefit of BPM-aware modeling in music understanding and mashup generation.

4.3. DSP Parameter Design Choices

To enable predictable, controllable, and learnable audio transitions, we design a compact set of DSP parameters that describe the entire mixing transformation. Our design follows three guiding principles:

Parameter	Symbol	Description	Training Role
Tempo and Beat Encoding			
Beats per minute	BPM	Estimated tempo of the audio; defines beat grid; not predicted.	DSP-only
Beat position encoding	e_{BPM}	Beat-level positional embedding fused into encoder features.	DSP-only
Transition Length Control			
Duration ratio	α	Controls the length of the mashup transition.	Model
Filter Parameters (Automatic)			
Low-stage HPF cutoff	HPF ₁	Removes low-end rumble from outgoing track.	Model
High-stage HPF cutoff	HPF ₂	Stronger low-end reduction during fade-out of track A.	Model
Low-stage LPF cutoff	LPF ₁	Darkens incoming track B in early transition.	Model
High-stage LPF cutoff	LPF ₂	Restores brightness as B fades in.	Model
Warm Equalization (Automatic)			
Low-band EQ gain	EQ _{low}	Boost or cut below 200 Hz.	Model
Mid-band EQ gain	EQ _{mid}	Emphasizes vocals/body (200–2000 Hz).	Model
High-band EQ gain	EQ _{high}	Controls harshness and presence (>2 kHz).	Model
Crossfade and Loudness			
Crossfade gain for A	$g_A(t)$	Time-varying outgoing gain curve with beat modulation.	DSP-only
Crossfade gain for B	$g_B(t)$	Complementary incoming gain curve.	DSP-only
Loudness envelope	$L(t)$	Combined duck–recover envelope with beat pumping.	DSP-only

Table 1. Summary of all DSP parameters and their roles. Some parameters are predicted by the model, while others are used internally by the DSP engine to ensure smooth audio transitions.

- **Interpretability:** The parameters correspond to standard operations used by human mixing engineers.
- **Smooth temporal evolution:** All parameters are designed to evolve across the transition window in a time-continuous or beat-aware way.
- **Predictability:** The full DSP parameter vector is compact and structured, making it feasible for a neural model to predict.

We incorporate filtering, equalization, crossfading, loudness envelopes, and global timing controls. Each parameter is automatically extracted from the audio using spectral analysis, BPM estimation, or psychoacoustic heuristics. Stochastic perturbations are also introduced to mimic human mixing variability and improve model generalization. Table 1 summarizes all DSP parameters used in our system.

4.4. BPM Aware Positional Encoding

Sinusoidal positional encoding provides both absolute and relative position signals, but it is not aligned with the rhythmic structure of music. In contrast, BPM defines a natural temporal coordinate system for beats, subdivisions, and phrase-level musical segments. Therefore, we introduce a

BPM-aware positional encoding that maps each frame into a continuous beat index before applying sinusoidal projections.

We split the sequence into two halves corresponding to song A and song B. Let T be the total number of frames and

$$T_{\frac{1}{2}} = \left\lfloor \frac{T}{2} \right\rfloor.$$

Given the sampling rate sr and the BPM values BPM_A and BPM_B , the corresponding beat periods (in seconds) are

$$P_A = \frac{60}{\text{BPM}_A}, \quad P_B = \frac{60}{\text{BPM}_B}.$$

For each frame index $i = 0, 1, \dots, T - 1$, the continuous beat index used in our positional encoding is computed as

$$\text{pos}_i = \begin{cases} \frac{i}{P_A}, & i < T_{\frac{1}{2}}, \\ \frac{i - T_{\frac{1}{2}}}{P_B}, & i \geq T_{\frac{1}{2}}. \end{cases}$$

This yields a frame-aligned beat index scaled by each song’s BPM.

Finally, we transform the beat index into a sinusoidal positional encoding. Let $\omega_j = 10000^{-2j/D}$ denote the standard logarithmic frequency spectrum. The BPM-aware positional encoding for frame i is then

$$\begin{aligned} \text{PE}_{i,2j} &= \sin(\text{pos}_i \omega_j), \\ \text{PE}_{i,2j+1} &= \cos(\text{pos}_i \omega_j), \quad j = 0, \dots, \frac{D}{2} - 1. \end{aligned}$$

This produces a sinusoidal embedding whose phase progression is aligned with the musical beat structure rather than the raw frame index as Figure 1.

5. Results

To evaluate the effectiveness of the proposed BPM Position Embedding, we conduct an ablation study with three model variants. The baseline model does not use any positional encoding (*No-PE*). We then compare (i) a model with standard vanilla positional encodings (*Vanilla-PE*) (Vaswani et al., 2017) and (ii) a model equipped with the proposed BPM Position Embedding (*BPM-PE*).

All experiments are carried out with a single Transformer block with 8 attention heads and hidden dimension $d \in \{32, 128, 512\}$. Given a pair of songs, we first use MERT to encode each audio track into a sequence of frame-level embeddings. These representations are concatenated and used as the input to the Transformer block. Each model is trained for 3 epochs with a batch size of 8 and a learning rate of 2×10^{-4} . We fix the random seed to 42 for all runs to ensure a fair comparison.

To isolate the effect of positional encoding, we construct a synthetic dataset by randomly concatenating segments from different tracks. Some concatenated segments are taken from the same track and placed before and after the boundary; in these cases, no tempo change occurs and the model is expected to output a default value. These confusing examples account for 15% of the training set. The training curves for all settings are shown in Figure 2.

As illustrated in Figure 2, when the model dimension is small ($d = 32$), there is little difference among the three variants, which suggests that the model capacity is insufficient to fully exploit BPM-related information. However, as the hidden dimension increases ($d = 128, 512$), the advantage of BPM-PE becomes increasingly pronounced, while the performance of Vanilla-PE gradually degrades.

6. Division of Labor

Hongyang Du. Proposed the central idea of incorporating BPM-aware modeling into the system to better align rhythmic structure during mixing. Designed and implemented the BPM Position Encoding module, including beat-index extraction and embedding injection into the MERT encoder.

Suggested and helped design model comparison baselines, including ablation variants with and without BPM encoding as well as alternative DSP-parameter prediction setups. Assisted in interpreting rhythmic-alignment results and guided the direction of the experimental analysis.

Ruijie Ye. Implemented the overall model training framework, including dataset loading, preprocessing, batching, and loss computation. Built the Transformer-based regression head that predicts DSP parameters from MERT+PE representations. Conducted all experiments, including training runs, ablation studies, evaluation of prediction accuracy, and qualitative listening tests. Analyzed model behavior and coordinated experimental iterations with the DSP engine design.

Yuehuan Qiu. Designed and implemented the complete DSP engine, including filtering, EQ, gain envelopes, loudness shaping, and full transition rendering. Developed the ground-truth automatic parameter generator, which computes HPF/LPF curves, EQ gains, and transition lengths from audio using deterministic DSP logic. Produced the ground-truth audio dataset used for training and evaluation, acting as the “ideal DJ” who provides perfect mixing supervision. Integrated model predictions into the DSP system to generate full mixed-audio demos.

7. Ethics

Deep learning is a natural fit for this problem because the quality of music blending depends on high-level factors such as tempo, rhythm, and style, which are difficult to capture with hand-crafted rules or simple signal-processing features. A representation model like MERT provides rich embeddings that already encode such information. By applying a deep model on top of these embeddings, we can learn a highly non-linear, end-to-end mapping from raw audio input to synthesis parameters, without explicitly designing features or symbolic music descriptors. This formulation is also easy to scale: as more training data becomes available, the same architecture can be trained on larger datasets to improve performance.

At the current stage, we quantify error and success by directly comparing the predicted synthesis parameters with the ground-truth parameters, e.g., using losses such as mean squared error. This choice makes optimization straightforward and fully differentiable, but it also introduces an important limitation. For many transitions there exist multiple parameter settings that can all lead to perceptually good blends, whereas our supervision only provides a single “reference” solution. As a result, the model is implicitly encouraged to imitate the specific blending style present in the dataset and may be penalized for producing alternative, yet musically valid, solutions. In other words, our

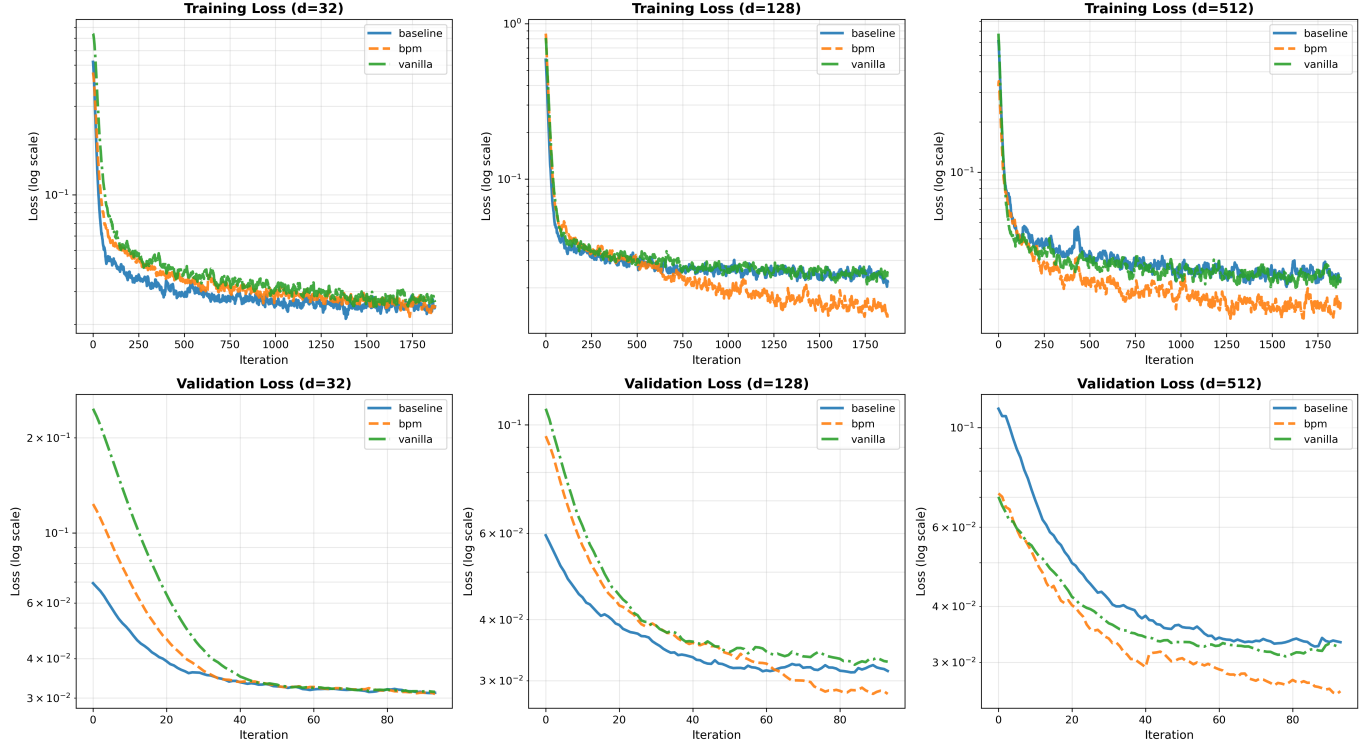


Figure 2. Training and validation loss curves for models with different positional encodings and hidden dimensions. The baseline model does not use any positional encoding (*baseline*), while the other variants use standard vanilla positional encoding (*vanilla*) and the proposed BPM Position Embedding (*bpm*). All models consist of a single Transformer block with 8 attention heads and hidden dimension $d \in \{32, 128, 512\}$. The top row shows training loss and the bottom row shows validation loss (both in log scale).

metric biases the model toward our preferred style rather than toward the full space of acceptable mixes. A more flexible evaluation based on perceptual scores or human preference—potentially combined with reinforcement learning instead of pure imitation learning—could give the model more freedom and better align the objective with subjective listening quality.

8. Reflection

The overall behavior of our model matches our expectations: since BPM is a key cue for DJ mixing, explicitly encoding BPM information leads to more consistent blending decisions. Our approach also evolved during the project. We initially aimed to build a model that directly generates audio waveforms, but found that this would require prohibitive computation and data. We therefore switched to a parameterized formulation and predict synthesis parameters instead of raw audio.

During the design of the BPM Position Embedding, we made several revisions. Our initial version caused the positional encoding to be largely ineffective, as the BPM signal

was not properly injected into the representation. After modifying how BPM is incorporated, we obtained an embedding that reliably encodes tempo information and improves downstream performance.

There are still important limitations. Training remains restricted to the parameter space, which can introduce bias in predictions, especially because parameter magnitudes vary widely. Although we apply normalization, this imbalance still affects optimization. A differentiable rendering pipeline—synthesizing audio from predicted parameters and computing loss in the signal domain—could unify the objective and reduce this bias. Furthermore, our current method performs pure imitation learning, reproducing a single mixing style instead of exploring the space of perceptually valid solutions. In future work, we plan to use a reward signal from an audio–language model or human preference scores and train with reinforcement learning, giving the model more flexibility while improving subjective listening quality.

References

- Baevski, A., Zhou, Y., Mohamed, A., and Auli, M. wav2vec 2.0: A framework for self-supervised learning of speech representations. *Advances in neural information processing systems*, 33:12449–12460, 2020.
- Defferrard, M., Benzi, K., Vandergheynst, P., and Bresson, X. FMA: A dataset for music analysis. In *18th International Society for Music Information Retrieval Conference (ISMIR)*, 2017. URL <https://arxiv.org/abs/1612.01840>.
- Hsu, W.-N., Bolte, B., Tsai, Y.-H. H., Lakhota, K., Salakhutdinov, R., and Mohamed, A. Hubert: Self-supervised speech representation learning by masked prediction of hidden units. *IEEE/ACM transactions on audio, speech, and language processing*, 29:3451–3460, 2021.
- Oppenheim, A. V. and Schaffer, R. W. *Discrete-Time Signal Processing*. Pearson, Upper Saddle River, NJ, 3rd edition, 2010.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Yizhi, L., Yuan, R., Zhang, G., Ma, Y., Chen, X., Yin, H., Xiao, C., Lin, C., Ragni, A., Benetos, E., et al. Mert: Acoustic music understanding model with large-scale self-supervised training. In *The Twelfth International Conference on Learning Representations*, 2023.